

K-means, Gaussian Mixtures, and EM (Lab 3)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, September 30, 2026

Instructor: Prof. Jongmin Lee

How to read this:

- Read it before the lab session. The lab will not repeat this material.
- Covered: clustering without labels, K-means with k-means++ seeding, Gaussian mixture models and EM, log-space computation (log-sum-exp), and model selection with BIC.
- Not covered: code and function usage. See the docstrings in `src/lab03.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy — grading and AI use — was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Implement K-means with k-means++ seeding, Lloyd iterations, the empty-cluster re-seeding rule, and multiple restarts. (Task 1)
2. Implement the E-step and M-step of a full-covariance Gaussian mixture in closed form, with the E-step computed in log space via log-sum-exp. (Task 2)
3. Verify on the trace that EM never decreases the log-likelihood. (Task 2, 4)
4. Count the free parameters of a full-covariance GMM and select the number of components with BIC. (Task 3)
5. Compare K-means and the GMM by ARI on an anisotropic 3D scene and explain why they disagree. (Task 4, report)

2 Background: a scan comes without labels

Every lab so far shipped an answer key. The appointments of Lab 2 carried no-show labels; the waiting-time data of the same lab came with its target column filled in. Freshly collected data carries no labels at all. A 3D scanner, for one, outputs nothing but coordinates. That is this lab's data: 1614 points sampled from the surfaces of an elongated box, a flat plate, a spherical shell, and a thin rod, merged into one cloud, and nobody tells you which point belongs to which object. Splitting the points into objects, that is, finding structure without labels, is *clustering*.

The central idea of this lab is that clustering is a chicken-and-egg problem. Imagine an auditorium where students from two classes sit mixed together and you must estimate each class's average height. If you knew who belongs to which class, the averages would be immediate. Conversely, if you knew the two averages, you could guess each student's class. Knowing neither, you alternate: assign memberships using rough averages, then recompute the averages from those memberships, and repeat. That alternation is K-means; soften the memberships into probabilities and it becomes EM.

In this lab you build both methods and run them on the same scene. The objects are deliberately elongated and flat, so K-means, which measures distance the same way in every direction, cuts the long box in two, while the GMM, which estimates a covariance per cluster, separates the four objects almost perfectly. Finally, you ask the data how many clusters there are in the first place (BIC). Grading checks that your update equations are right (the hidden tests recompute one E-step and one M-step from your own responsibilities) and that you can interpret the outcome.

The four steps of Lab 3:

- **K-means:** choose seeds with `k-means++` and refine with Lloyd iterations. (Task 1)
- **GMM with EM:** split memberships into probabilities (E) and update parameters with weighted averages (M). (Task 2)
- **Model selection:** fit one GMM per candidate k and take the smallest BIC. (Task 3)
- **The comparison:** score both methods by ARI against the ground-truth labels, and record the EM monotonicity check and the BIC curve. (Task 4)

3 Core concepts

3.1 K-means and k-means++ seeding

K-means picks centers $c_1, \dots, c_k$ and assignments z_i to minimize the *inertia*, the sum of squared distances between points and their assigned centers.

$$J = \sum_{i=1}^N \|x_i - c_{z_i}\|^2$$

Lloyd iterations (Task 1):

- **Assign:** attach each point to its nearest center, $z_i = \arg \min_j \|x_i - c_j\|^2$.
- **Empty-cluster re-seeding:** if a cluster ends up with no points, move the point farthest from its own center into that cluster. Left alone, the next update divides by zero and the center becomes NaN. Order and tie rules are fixed in the docstring.
- **Update:** move each center to the mean of its members.
- **Converge:** stop when the assignment repeats the previous iteration. Both steps lower J , so the loop always terminates.

k-means++ seeding and restarts:

- **The problem with uniform seeding:** picking k arbitrary points often puts several seeds in a large cluster and none in a small one. Lloyd is a local search and frequently cannot repair that mistake.
- **D^2 sampling:** draw the first seed uniformly; draw each next seed with probability proportional to D^2 , the squared distance to the nearest seed chosen so far. Far-away regions become likely seed locations (Arthur and Vassilvitskii, 2007).
- **Multiple restarts:** seeding is random, so run `N_INIT=4` independent restarts and keep the one with the smallest inertia.
- **One generator, one call order:** all randomness comes from the `rng` argument, never from `np.random.*`, so the same seed always gives the same result. The docstring also fixes the order of the draws (one `rng.integers` call, then one `rng.choice(n,`

`p=d2/d2.sum()`) call per further seed). Follow it and your numbers reproduce the reference values; a different order is not penalized, but your results will differ from everyone else's and are harder to check.

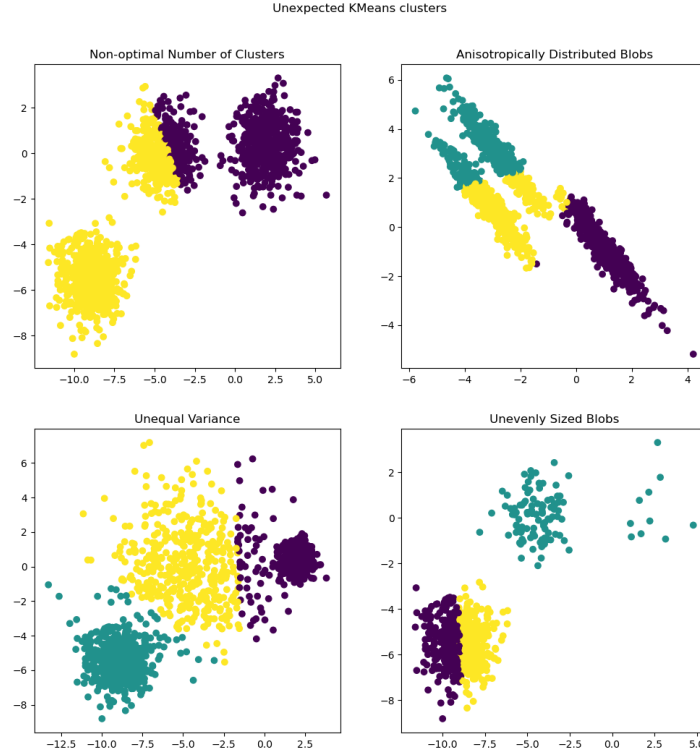


Figure 1. Four ways K-means goes wrong. Clockwise from top left: a wrong choice of k , anisotropic clusters, unevenly sized clusters, and unequal variances, each colored by the K-means assignment. The top-right panel is exactly this lab's situation: the isotropic distance slices elongated clusters across and mixes them with their neighbors. Source: scikit-learn developers, "Demonstration of k-means assumptions," scikit-learn 1.9 documentation (scikit-learn.org), BSD-3, 2026.

Assumptions and limits:

- Measuring distance identically in all directions implicitly assumes round, similar-sized clusters. This lab's box (covariance eigenvalue ratio about 22) and plate (about 45) violate that assumption exactly the way Figure 1's top-right panel does.
- Inertia always decreases as k grows, so inertia alone cannot choose k . BIC (§3.4) fills that gap.

3.2 Gaussian mixtures and EM

A *Gaussian mixture model* treats the data as drawn from k Gaussians with mixing weights w_j .

$$p(x) = \sum_{j=1}^k w_j \mathcal{N}(x \mid \mu_j, \Sigma_j)$$

With memberships unknown, no closed form maximizes the log-likelihood $\sum_i \log p(x_i)$ directly. EM (*Expectation-Maximization*) follows the auditorium analogy literally: it alternates membership estimation and parameter estimation (Figure 2).

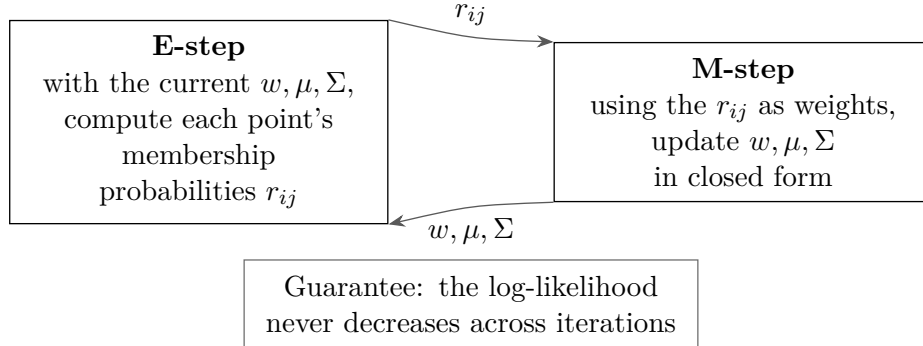


Figure 2. The EM alternation. Knowing memberships makes the parameters immediate; knowing parameters makes memberships estimable as probabilities. Alternating the two climbs the log-likelihood monotonically to a fixed point. This lab runs the loop for a fixed GMM_MAX_ITER= 80 iterations.

The two closed-form steps (Task 2):

- **E-step (responsibilities):** $r_{ij} = \frac{w_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_l w_l \mathcal{N}(x_i | \mu_l, \Sigma_l)}$, a membership distribution per point (each row sums to 1).
- **M-step (weighted updates):** with $N_j = \sum_i r_{ij}$: $w_j = N_j/N$, $\mu_j = \frac{1}{N_j} \sum_i r_{ij} x_i$, $\Sigma_j = \frac{1}{N_j} \sum_i r_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top + \text{reg } I$. Instead of averaging only a class's own students, every student contributes in proportion to their membership probability.
- **Initialization:** run K-means (Task 1) once and start μ, w, Σ from its centers and assignment. EM is a local search too; the initialization decides where it lands.

The E-step lives in log space (log-sum-exp):

- **Why:** log-densities of far components reach several hundred below zero on scenes like this lab's and the -10^7 range on a hidden validation scene. float64's exp underflows to 0 below about -745 , so exponentiating densities first makes both numerator and denominator 0 for some points: responsibilities become 0/0 and the log-likelihood becomes $\log 0$.
- **The fix:** subtract each row's maximum before exponentiating. The largest term becomes $\exp(0) = 1$, which can never underflow.
- **The covariance floor:** a perfectly flat cluster (like the plate) has a singular sample covariance with no inverse. Adding $\text{reg } I$ ($\text{reg} = 10^{-6}$) at initialization and after **every** M-step keeps every covariance invertible.

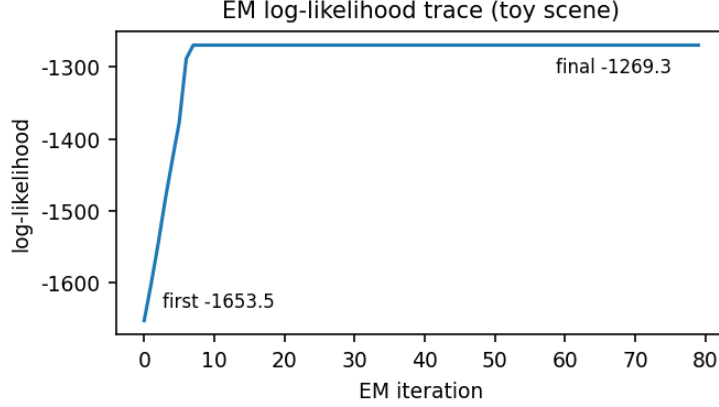


Figure 3. The EM log-likelihood trace on the toy scene of Figure 5. Starting from the K-means initialization at -1653.5 , the trace reaches the -1269.3 plateau within about 7 iterations. Over all 80 iterations the smallest consecutive change is about -9×10^{-10} : the bias of the `reg I` term, which makes the M-step slightly non-optimal; monotonicity holds to within that size. Your trace on the lab’s scene has the same shape with its own numbers. Generated by the author.

Assumptions and limits:

- The monotonicity guarantee is for the exact M-step. Adding `reg I` makes the update slightly non-optimal, so microscopic decreases are possible in theory; the -9×10^{-10} in Figure 3’s trace is that size. The check in `results.json` (`ll_min_delta`) treats this level as normal.
- The fixed point is a local optimum. Where EM lands is decided by the K-means initialization, which in turn reduces its own risk with restarts.

3.3 K-means is a special case of EM

The two algorithms are two versions of the same loop (Table 1). Fix every covariance to the isotropic $\Sigma_j = \varepsilon I$ and take $\varepsilon \rightarrow 0$: the responsibilities harden into 0/1 on the nearest component, and EM’s two steps collapse exactly into Lloyd’s assign and update.

Table 1. Two versions of one loop. The two columns differ only in whether the covariance is estimated or frozen; that difference is the whole ARI gap between the two panels of Figure 5.

Aspect	K-means	EM for a GMM
Membership	0/1 (nearest center)	probabilities r_{ij} (rows sum to 1)
Cluster shape	isotropic (all directions equal)	full covariance Σ_j
Update	mean of members	responsibility-weighted mean and covariance
Objective	inertia J	log-likelihood

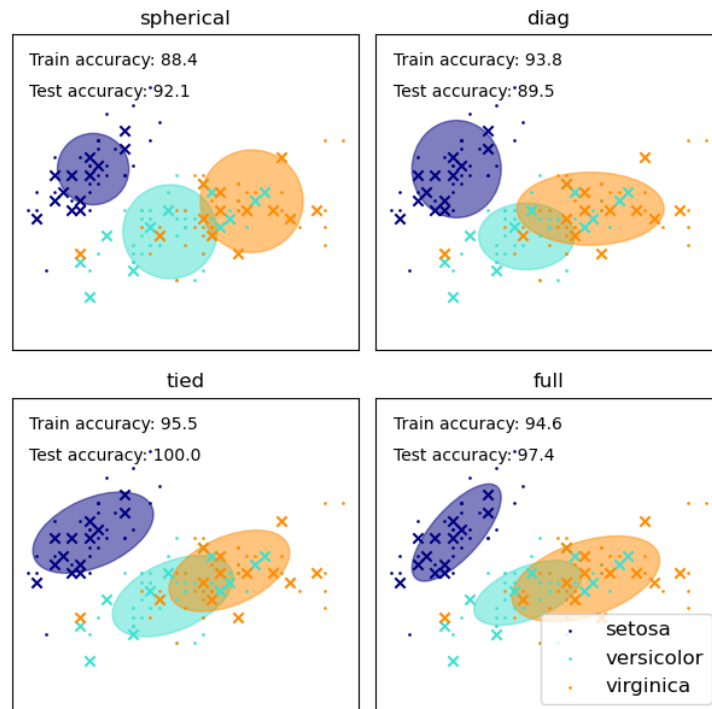


Figure 4. Covariance type decides cluster shape. Gaussian mixtures with one component per iris species. Restricting the covariance to spherical gives circles; diagonal gives axis-aligned ellipses; tied shares one covariance across components, so every ellipse has the same shape; full lets the ellipse tilt along the data. Task 2 of this lab implements full. Source: scikit-learn developers, “GMM covariances,” scikit-learn 1.9 documentation (scikit-learn.org), BSD-3, 2026.

What happens on a toy scene:

- **The scene:** 600 points in 2D — a long diagonal bar (260 points, about 7.5 long and 0.3 wide), a flat horizontal ellipse (200), and a round blob (140) whose center sits only 2.4 from the bar’s center (Figure 5). It is not the lab’s data; it is the same mechanism in two dimensions.
- **K-means (ARI 0.54):** the isotropic Voronoi partition cuts the bar into two pieces (113 and 147 points) and merges the piece nearer the blob with all 140 blob points (Figure 5, left). Only the flat ellipse comes out clean.
- **GMM (ARI 1.00):** the bar component’s covariance stretches along the diagonal (Figure 6) and reclaims the whole bar; all three clusters are recovered exactly (Figure 5, right).
- **How to read it:** the entire ARI gap comes from the difference in shape assumptions. Were every cluster a disc, the two methods would give nearly the same answer. The lab’s 3D scene tells the same story with four objects; run it and read your own numbers off `results.json`.

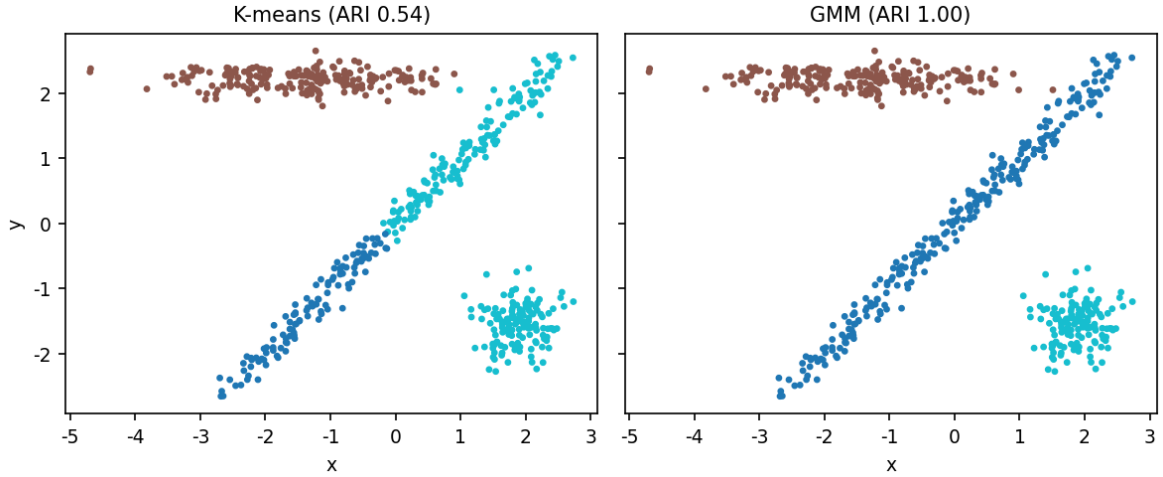


Figure 5. One toy scene, two assignments. Colors are each algorithm’s cluster assignment. K-means (left) cuts the long bar and merges the piece nearer the blob into the blob’s cluster. The GMM (right) separates the three clusters along their true boundaries. Generated by the author.

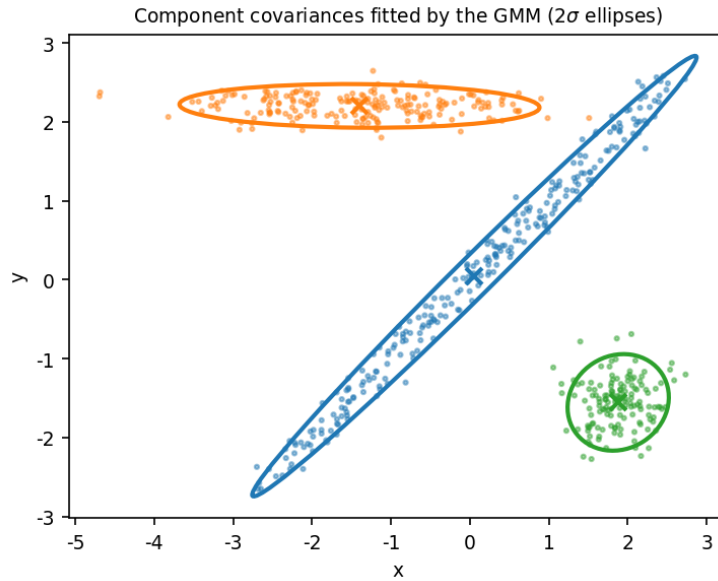


Figure 6. The component covariances fitted by the GMM on the toy scene. Each component’s 2σ ellipse. The bar (blue) and the flat ellipse (orange) stretch, the blob (green) stays round: covariances shaped like the clusters themselves, the full case of Figure 4. Generated by the author.

Assumptions and limits:

- The GMM’s flexibility ends at ellipsoids. Bent clusters such as crescents or spirals fit no single Gaussian, and both methods fail on them; that data belongs to other families such as spectral clustering.
- ARI is an evaluation metric that needs ground-truth labels. This lab’s `labels` are for evaluation only and must never enter the fitting (`kmeans`, `gmm_em`, `select_k`). The hidden tests rerun the experiment with shuffled labels and require every fit-derived number to be unchanged.

3.4 How many clusters: BIC

So far $k = 4$ was given, but in practice k itself is unknown. Goodness of fit alone cannot answer: more components always fit better (on the toy scene the final log-likelihood climbs from -1949 at $k = 2$ to -1225 at $k = 8$). The BIC (*Bayesian information criterion*) resolves this by charging a complexity penalty.

$$\text{BIC} = -2\ell + p \log N \quad (\ell : \text{log-likelihood}, p : \text{number of free parameters})$$

Counting parameters (Task 3):

- **Means:** $k \times d$. **Covariances:** symmetric, so $d(d+1)/2$ per component, $k \times d(d+1)/2$ in total. **Weights:** they sum to 1, so $k - 1$.
- **For the toy scene:** $d = 2$, $k = 3$ gives $6 + 9 + 2 = 17$. For this lab ($d = 3$) count them yourself; the docstring gives the formula.
- **The size of the penalty:** on the toy scene each extra component adds 6 parameters, hence a penalty of $6 \log 600 \approx 38.4$. Going from $k = 3$ to 4 buys only $2\Delta\ell \approx 30.1$ of fit, which does not cover the penalty, so the BIC curve bottoms out at 3 (Figure 7). On the lab's scene the penalty per component is $10 \log N$; compare it with the fit gain you observe.

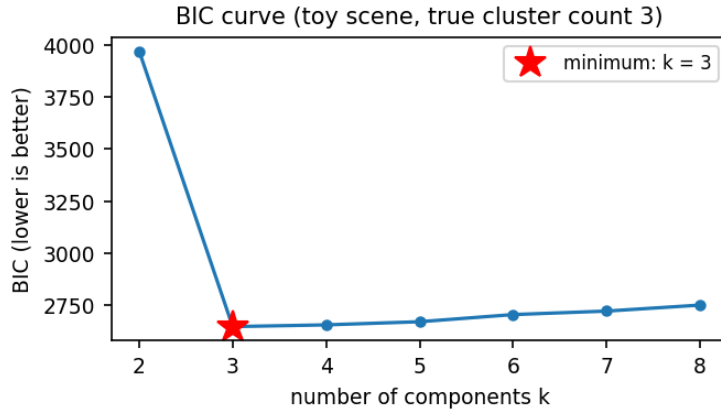


Figure 7. The BIC curve on the toy scene. From $k = 2$ to 3 the BIC falls steeply as the missing component gets filled in (a drop of about 1320); past 3 the penalty outweighs the gain and the curve climbs gently (8.3 up to 4). The minimum sits at the true cluster count, 3. Generated by the author.

Assumptions and limits:

- BIC is a large-sample approximation and tends to recover the true k when the data really is a Gaussian mixture. This lab's objects are surface points, not exact Gaussians, so the margin at the true k is not unlimited (8.3 on the toy scene): make the objects more shell-like and the minimum drifts upward.
- `select_k` refits EM per candidate, inheriting each fit's local-optimum problem. That is why the candidate order and the generator convention are fixed in the docstring: it makes the curve reproducible.

4 Worked example: one EM iteration on four 1D points

Table 2 gives four points and initial parameters; we run the E-step, the M-step, and the log-likelihood to the end. The lab implements the 3D full-covariance version, but the procedure is identical; in 1D the covariance is just a variance. Every number was computed with code and transcribed, rounded to 4 decimals.

Table 2. Data and initial values for the worked example. Four points, two components. The initial weights and variances are equal, so the responsibilities reduce to comparing two exponentials.

Item	Value
Data $x_1, \dots, x_4$	0, 1, 3, 5
Initial means μ_1, μ_2	1, 4
Initial variances σ_1^2, σ_2^2	1, 1
Initial weights w_1, w_2	0.5, 0.5

E-step: responsibilities:

1. With equal weights and variances, $r_{i1} = \frac{e^{-(x_i-1)^2/2}}{e^{-(x_i-1)^2/2} + e^{-(x_i-4)^2/2}}$.
2. $x = 0$: $e^{-0.5} = 0.6065$ vs $e^{-8} = 0.0003 \Rightarrow r_{11} = 0.9994$.
3. $x = 1$: $e^0 = 1$ vs $e^{-4.5} = 0.0111 \Rightarrow r_{21} = 0.9890$.
4. $x = 3$: $e^{-2} = 0.1353$ vs $e^{-0.5} = 0.6065 \Rightarrow r_{31} = \mathbf{0.1824}$. Only the point caught between the components gets a genuinely split responsibility.
5. $x = 5$: $e^{-8} = 0.0003$ vs $e^{-0.5} = 0.6065 \Rightarrow r_{41} = 0.0006$.
6. Log-likelihood of the current parameters: $\ell = -7.7348$.

M-step: weighted updates:

1. Effective counts: $N_1 = 0.9994 + 0.9890 + 0.1824 + 0.0006 = 2.1714$, $N_2 = 1.8286$.
2. Weights: $w'_1 = 2.1714/4 = 0.5429$, $w'_2 = 0.4571$.
3. Means: $\mu'_1 = (0.9994 \cdot 0 + 0.9890 \cdot 1 + 0.1824 \cdot 3 + 0.0006 \cdot 5) / 2.1714 = 0.7088$, $\mu'_2 = 4.0802$.
4. Variances: $\sigma_1'^2 = 0.7156$, $\sigma_2'^2 = 1.0462$ (adding `reg = 10-6` changes no fourth decimal).
5. Log-likelihood after the update: $\ell' = -7.4756$. The change is $+0.2592 > 0$: monotonicity holds through the final step.

One K-means iteration from the same points and centers:

- Assign: $0, 1 \rightarrow$ center 1, $3, 5 \rightarrow$ center 2 ($|3 - 1| = 2 > |3 - 4| = 1$).
- Update: $c'_1 = (0 + 1)/2 = 0.5$, $c'_2 = (3 + 5)/2 = 4$.
- The two methods differ at exactly one point, $x = 3$. K-means hands it entirely to component 2 (as if r_{31} froze to 0), while EM keeps 18.24% of it in component 1; that share pulls μ'_1 up to 0.7088, above K-means's $c'_1 = 0.5$. Splitting boundary points probabilistically is the entire hard-vs-soft difference.

Spec: the E-step in log space (Task 2 docstring)

For every i, j build $\text{logp}[i, j] = \log w_j + \log \mathcal{N}(x_i \mid \mu_j, \Sigma_j)$, and compute the per-point normalizer with log-sum-exp: $m_i = \max_j \text{logp}[i, j]$, $\text{lse}_i = m_i + \log \sum_j \exp(\text{logp}[i, j] - m_i)$, $r_{ij} = \exp(\text{logp}[i, j] - \text{lse}_i)$. Append $\ell = \sum_i \text{lse}_i$ to the trace (the log-likelihood of the CURRENT parameters, before this iteration's M-step).

- If this document ever seems to disagree with the spec wording, **the spec wins**.

5 Common mistakes

1. **Computing the E-step in probability space.** Exponentiating densities first and normalizing later makes numerator and denominator both 0 wherever log-densities dip below -745 : responsibilities become NaN. It looks fine on the public data and explodes on the hidden wide-dynamic-range scene. The log-sum-exp of §3.2 is the only fix.
2. **Not normalizing responsibility rows.** If rows of r_{ij} do not sum to 1, the effective counts N_j and everything after them are wrong. The tests check the row sums and recompute one E-step and one M-step from your own responsibilities.
3. **Computing the covariance with the pre-update means.** The M-step Σ_j uses the μ_j just updated in the same M-step (the docstring says so). The pre-update means pass every public test and fail the hidden test that recomputes the M-step from your own responsibilities.
4. **Adding reg only at initialization.** The first iteration runs, but the M-step regenerates a singular covariance for a flat cluster and the next E-step dies. The spec adds `reg I` after **every** M-step.
5. **Randomness outside rng, or uniform seeding.** A `np.random.*` call anywhere makes the same seed give different results, and the hidden tests require determinism. Replacing D^2 sampling with uniform draws is plain K-means: it misses a cluster on the hidden eight-cluster scene. Reordering the `rng` calls is not penalized, but it changes your numbers.
6. **Ignoring empty clusters.** The mean of a memberless cluster is $0/0$. The re-seeding rule in the docstring is what keeps k clusters alive even in boundary cases with many duplicate points.
7. **Using the ground-truth labels in fitting.** `labels` exist solely for the evaluation step of `run_experiment` (the two ARI calls). If labels leak into initialization or the choice of k , the hidden test that reruns the experiment with shuffled labels exposes it immediately. Hard-coding numbers you read off the shipped data fails on hidden data for the same reason.

6 Mapping to the assignment

Table 3. Tasks versus concept sections. Which section each task builds on and what grading looks at. Check this once before writing the report.

Lab task	Section	What grading checks
Task 1. <code>kmeans</code>	§3.1	D^2 seeding, empty-cluster rule, determinism, self-consistent centers and inertia
Task 2. <code>gmm_em</code>	§3.2, §4	log-sum-exp, closed-form M-step, reg, the trace
Task 3. <code>bic</code> , <code>select_k</code>	§3.4	BIC formula, parameter count, one fit per candidate
Task 4. <code>run_experiment</code>	§3.3, §3.4	label isolation, determinism, no hard-coded numbers
Report Figures 1–3	§3.2–3.4	assignment comparison, EM trace, BIC curve, and their reading

7 Check yourself

1. What does D^2 sampling buy over uniform seeding, and why are multiple restarts still necessary?
2. When some log-density is -800 , what exactly happens in a probability-space E-step, and why does subtracting the row maximum prevent it?
3. Explain in one line each what the E-step and the M-step contribute to EM never decreasing the log-likelihood. Why can a trace still show a decrease of order 10^{-9} ?
4. Which restriction on the covariances, and which limit, turns EM into K-means? Why does that restriction cost almost half the ARI on the toy scene of Figure 5, and what change to the scene would make the two methods agree?
5. The log-likelihood is best at the largest k , yet BIC picks the true count. What does each of the two terms measure, and how do you compute the per-component penalty for this lab’s scene ($d = 3$, $N = 1614$)?

These questions are oral-check material. You must be able to explain every line you submit.

8 Further reading

- D. Arthur and S. Vassilvitskii, “k-means++: The Advantages of Careful Seeding,” *SODA*, 2007 (the original D^2 seeding paper, with expected approximation guarantees) <https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf>
- C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Chapter 9 (mixture models and EM, with the monotonicity proof) <https://www.microsoft.com/en-us/research/people/cmbishop/prml-book/>
- scikit-learn User Guide, “Clustering” and “Gaussian mixture models” (the menu of practical implementations and their trade-offs) <https://scikit-learn.org/stable/modules/clustering.html>